UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2017/2018

SEPTEMBER EXAMINATION

**UECS3253 WIRELESS APPLICATION DEVELOPMENT**

TUESDAY, 19 SEPTEMBER 2017 TIME : 2.00 PM – 4.00 PM (2 HOURS)

BACHELOR OF SCIENCE (HONS) SOFTWARE ENGINEERING

**Instruction to Candidates :**

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

**Section A:** Answer ALL questions.

**Section B:** Answer ONE (1) question only.

Each question carries 25 marks.

**Note:** Should a candidate answer more than ONE (1) question in Section B, marks will only be awarded for the FIRST question in Section B in the order the candidate submits the answer.

Answer the questions in the answer booklet provided.

**UECS3253 WIRELESS APPLICATION DEVELOPMENT**

## SECTION A

This section consists of **THREE (3)** questions. Answer ALL questions.

Q1. (a) (i) Explain what an *implicit intent* is. (2 marks)

(ii) Explain using an example how *implicit intents* are used in Android apps. You do NOT need to use any source code in your explanation. (3 marks)

(b) Explain the purpose of specifying a minimum API Level in an Android development project. (4 marks)

(c) Complete the missing code in the **AndroidManifest.xml** file in Figure 1.1 based on the following specifications:

(i) The app requires access to the Internet. (4 marks)

(ii) The **MainActivity** is the launcher activity. (4 marks)

(iii) The parent activity of **ViewRecordActivity** is the **MainActivity**. (4 marks)

(iv) The **ComposeActivity** may receive an **ACTION_SEND** *implicit intent.* (4 marks)

Q1. (c) (Continued)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.jamesooi.jsondemo">

      <!--
       !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
       !                           !
       ! MISSING CODE (i)          !
       !                           !
       !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
      -->

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name"
            android:theme="@style/AppTheme.NoActionBar">
                  <!--
                   !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
                   !                           !
                   ! MISSING CODE (ii)         !
                   !                           !
                   !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
                  -->
        </activity>
        <activity android:name=".ViewRecordActivity"
            android:parentActivityName=".MainActivity"
            >
                  <!--
                   !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
                   !                           !
                   ! MISSING CODE (iii)        !
                   !                           !
                   !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
                  -->
        </activity>
        <activity android:name=".ComposeActivity">
                  <!--
                   !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
                   !                           !
                   ! MISSING CODE (iv)         !
                   !                           !
                   !!!!!!!!!!!!!!!!!!!!!!!!!!!!!
                  -->
        </activity>
    </application>
</manifest>
```

**Figure 1.1**

[Total : 25 marks]

Q2. Table 2.1 shows an extract of sample data of a table named **customer** of an SQLite database to maintain customer information used in an Android app.

| id | name | email | phone |
|---|---|---|---|
| 1 | Chia Kim Hooi | khchia@geemail.com | 012-4027453 |
| 2 | Ng Pei Li | ngpeili@geemail.com | 016-2972977 |
| 3 | Foo Yoke Wai | fooyw@coldmail.com | 012-5578681 |
| 4 | Ang Chee Wan | angcw@yahuu.com | 017-4895111 |
| 5 | Lim Li Li | limlili@livemail.my | 012-6166384 |
| 6 | Christopher Ananda | chris@geemail.com | 017-5623618 |
| 7 | Mok Sook Chen | chen@geemail.com | 012-4023210 |

**Table 2.1**

A contact class named **CustomerDbContract** has been defined as shown in Figure 2.2 and the **Customer** class is as shown in the class diagram in Figure 2.3.

```
public class CustomerDbContract {

private CustomerDbContract() {}

    public static class CustomerEntry implements BaseColumns {
        public static final String TABLE_NAME = "customer";
        public static final String COLUMN_NAME_NAME = "name";
        public static final String COLUMN_NAME_EMAIL = "email";
        public static final String COLUMN_NAME_PHONE = "phone";
    }
}
```

**Figure 2.2**

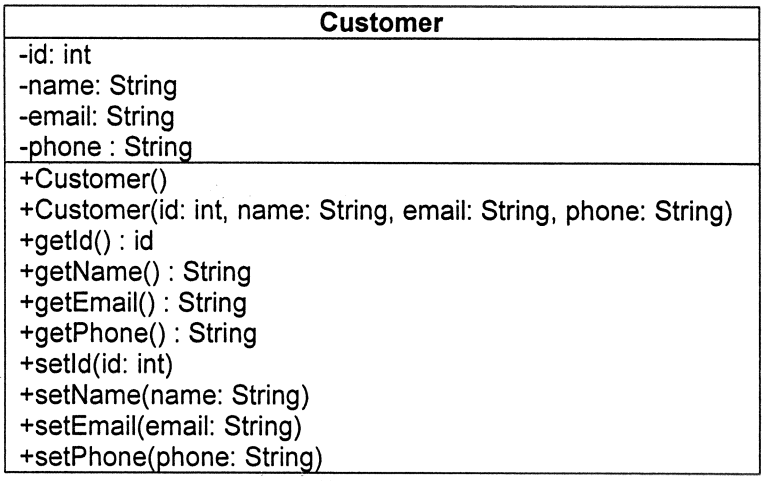


**Figure 2.3**

Q2. (Continued)

(a) Develop a subclass of **SQLiteOpenHelper** named **CustomerDbHelper** by completing the following missing code in Figure 2.4 based on the following questions.

```
public class CustomerDbHelper extends SQLiteOpenHelper {
    public static final int DATABASE_VERSION = 1;
    public static final String DATABASE_NAME = "customer.db";

            /*************************
             *                       *
             * MISSING CODE (i)      *
             *                       *
             *************************/

    public CustomerDbHelper(Context context) {
            /*************************
             *                       *
             * MISSING CODE (ii)     *
             *                       *
             *************************/
    }

    public void onCreate(SQLiteDatabase db) {
            /*************************
             *                       *
             * MISSING CODE (iii)    *
             *                       *
             *************************/
    }

    public void onUpgrade(SQLiteDatabase db,
      int oldVersion, int newVersion) {}
}
```

**Figure 2.4**

(i) A class constant named **SQL_CREATE_ENTRIES** containing SQL command to create the table. (5 marks)

(ii) Java code for the **CustomerDbHelper** constructor. (2 marks)

(iii) Java code for the **onCreate** method. (2 marks)

(b) An instance of the **Customer** class named **customer** contains updated data about a customer. Develop Java code segment to update the abovementioned customer data into the **customer** table. (7 marks)

(c) Develop Java code segment to retrieve all rows from the **customer** table, sorted by the **name** attribute in ascending order and construct an **ArrayList** of **Customer** objects to hold the data retrieved. (9 marks)

[Total : 25 marks]

Q3. An image gallery Android app displays a list of thumbnails on the screen. Clicking on a thumbnail displays the selected full-size image and the image meta-data on the screen. In this app, the data is pulled from an HTTP server over the public Internet.

Two **AsyncTask** classes are used by the abovementioned app to pull the required data from the HTTP server. The **DownloadImageTask** is an **AsyncTask** class that pulls the image bitmap from the server. The **DownloadJSONTask** is an **AsyncTask** class that pulls the meta-data of all images from the server. Figure 3.1 shows a sample of the data pulled from the server.

```
[
 {"file":"gamcheon.jpg",
  "thumbnail":"tn.gamcheon.jpg",
  "caption":"감천문화마을 Gamcheon Culture Village",
  "date":1495497600000
},
 {"file":"yonggungsa.jpg",
  "thumbnail":"tn.yonggungsa.jpg",
  "caption":"해동 용궁사 Haedong Yonggungsa Temple",
  "date":1495411200000
 },
 {"file":"changdeokgung.jpg",
  "thumbnail":"tn.changdeokgung.jpg",
  "caption":"창덕궁 Changdeokgung 昌德宮",
  "date":1495756800000
 },
 {"file":"gyeongbokgung.jpg",
  "thumbnail":"tn.gyeongbokgung.jpg",
  "caption":"경복궁 Gyeongbokgung 景福宮",
  "date":1495843200000
 }
]
```

**Figure 3.1**

Figure 3.2 shows the class diagram of the class named **Image** which is used to represent an image meta-data in this app.

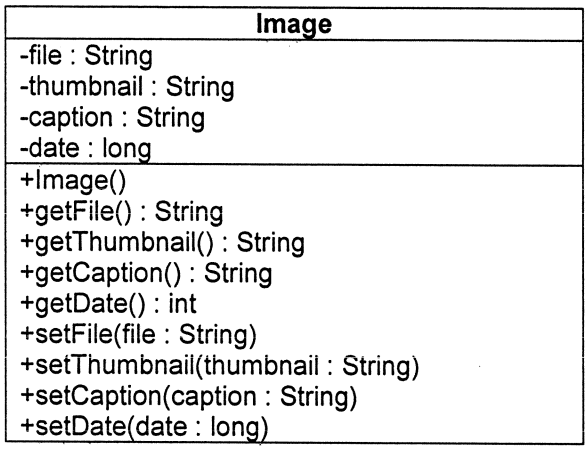


**Figure 3.2**

Q3. (Continued)

(a) Develop the **DownloadJSONTask** class by completing the following missing code in Figure 3.3:

(i) Makes a HTTPS connection using GET request to the first URL specified by the parameter **urls** and pulls the JSON data. Returns a **JSONArray** object. (10 marks)

(ii) Parses the **JSONArray** to build a **ArrayList** of **Image** objects. (5 marks)

```
private class DownloadJSONTask
  extends AsyncTask<String, Void, JSONArray> {
    @Override
    protected JSONArray doInBackground(String... urls) {
            /**********************************
             *                                *
             * MISSING CODE (i)               *
             *                                *
             **********************************/
    }

    @Override
    protected void onPostExecute(JSONArray jsonArray) {
            /**********************************
             *                                *
             * MISSING CODE (ii)              *
             *                                *
             **********************************/
    }
}
```

**Figure 3.3**

(b) Develop Java code segment to pull the data from the input stream as a **Bitmap** object to decode data from the **InputStream** into **Bitmap**. (4 marks)

(c) Develop Java code segment to begin execution of the **DownloadJSONTask**. Assume that the URL is specified by a class constant named **JSON_URL**. Ensure that your code checks for the availability of Internet connection before downloading. If no connection is available, display an error message using a **Toast**. (6 marks)

[Total : 25 marks]

## SECTION B

This section consists of **TWO (2)** questions. Answer **ONE (1)** question only.

Q4. Figure 4.1 shows an activity for configuring app settings of an Android Wi-Fi analyser app. The **id** attributes of the view components in the activity are labelled, if applicable.

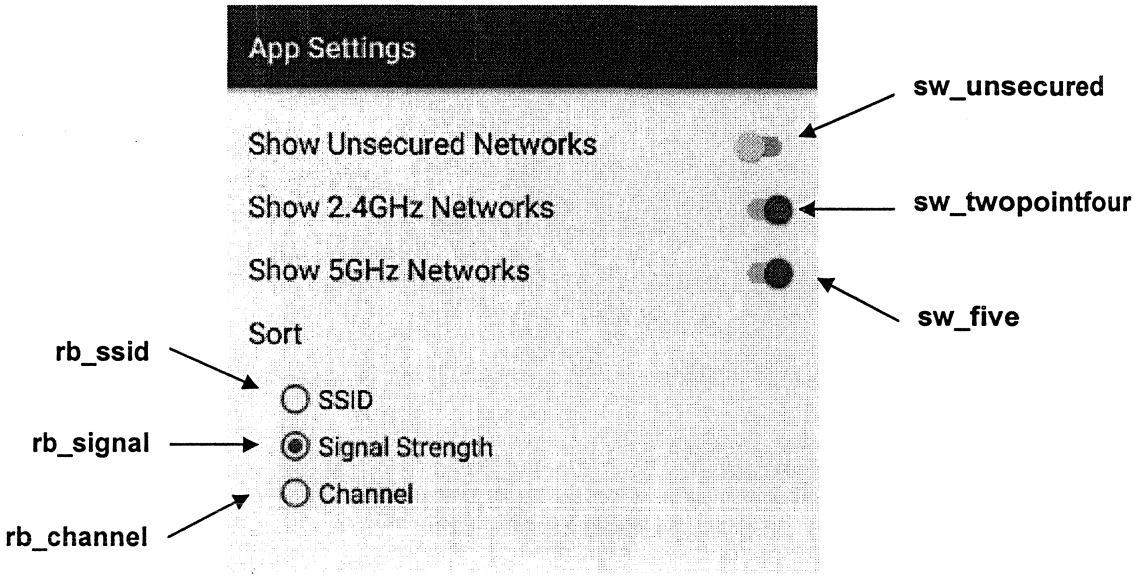


**Figure 4.1**

Figure 4.2 shows the class diagram of a Java class named **AppSettings** which will be used to represent the settings of this app.

| **AppSettings** |
|---|
| -showUnsecured: boolean<br>-show2point4GHz: boolean<br>-show5GHz: boolean<br>-sort : String |
| +AppSettings()<br>+isShowUnsecured() : boolean<br>+isShow2point4GHz() : boolean<br>+isShow5GHz() : boolean<br>+getSort() : String<br>+setShowUnsecured(showUnsecured: boolean)<br>+setShow2point4GHz(show2point4GHz: boolean)<br>+setShow5GH(show5GHz: boolean)<br>+setSort(sort: String) |

**Figure 4.2**

Q4. (Continued)

(a) Construct XML code segments to define the following views.

(i) The **Switch** view with the **id** ***sw_five***. (3 marks)

(ii) The **RadioGroup** view group and all the three **RadioButton** views with the **id** ***rb_ssid***, ***rb_signal*** and ***rb_channel*** respectively. All **RadioButton** views have the **onClick** attribute set to **onRadioButtonClicked**. (7 marks)

(b) Suppose that an instance of **AppSettings** named **settings** is used to maintain the current state of the app settings. When one of the **Switch** views is toggled, it sets the corresponding data field of **settings**. Similarly, when one of the **RadioButton** views is clicked, it sets the corresponding data field of **settings.**

Build Java code for the **save** method to implement the above functionality. (9 marks)

(c) Build Java code segment to set the **checked** attributes of all **Switch** and **RadioButton** views in accordance to the corresponding states of the data fields of an instance of **AppSettings** named **settings**. (6 marks)

[Total : 25 marks]

Q5. Figure 5.1 shows an activity of an Android app for Klang Valley metro railway system. In this activity, it lists the list of station names and ID of a railway line within a **ListView** view. Each item consists of two **TextView** views.

**Figure 5.1**

(a) Construct XML code segments to define the following layout files.

(i) The layout **activity_main** consisting of a **ListView** view with the **id** ***list_stations***. (3 marks)

(ii) The layout **item_station** consists of two **TextView** views nested within a **LinearLayout** view group. The **TextView** views are assigned with the **id** ***item_name*** and ***item_id***. The ***item_name*** view displays text with the size **20sp** and in **bold** style while the ***item_id*** view displays text with the size **18sp.** (7 marks)

Q5. (Continued)

(b) Develop a subclass of **ArrayAdapter** named **StationAdapter** by completing the missing code shown in Figure 5.2. The **StationAdapter** extends the **ArrayAdapter** class to bridge between **AdapterView** and the underlying data represented by an **ArrayList** of **Station** objects.

```
public class StationAdapter
        extends ArrayAdapter<Station> {

    private ArrayList<Station> data;
    Context mContext;

    private static class ViewHolder {
        TextView txtName;
        TextView txtID;
    }

    public StationAdapter(ArrayList<Station> data,
                                Context context) {
        super(context, R.layout.item_station, data);
        this.data = data;
        this.mContext = context;
    }

    @Override
    public View getView(int position, View convertView,
                        ViewGroup parent) {
        Station station = getItem(position);
        ViewHolder viewHolder;
        final View result;

        if(convertView == null) {
            viewHolder = new ViewHolder();
            LayoutInflater inflater =
                  LayoutInflater.from(getContext());


            /*************************
             *                       *
             * MISSING CODE          *
             *                       *
             *                       *
             *************************/
        }
        else {
            viewHolder =
                  (ViewHolder)convertView.getTag();
        }

        viewHolder.txtName.setText(station.getName());
        viewHolder.txtID.setText(station.getId());

        return convertView;
    }
}
```

**Figure 5.2**

(7 marks)

Q5. (Continued)

(c) Build Java code for the **AdapterView.OnItemClickListener** in the **MainActivity** to handle the event when an item in the **ListView** view is clicked by the user. The listener class must implement the following method:

```
public void onItemClick(
      AdapterView<?> adapterView,
      View view,
      int position,
      long id);
```

Clicking on an item starts a new activity named **DisplayStationActivity** to display the information of the specified station. The current activity sends a **Station** object to the **DisplayStationActivity** as an extra.

Assume that the **Station** object implements the **Serializable** interface.

(8 marks)
[Total : 25 marks]

---